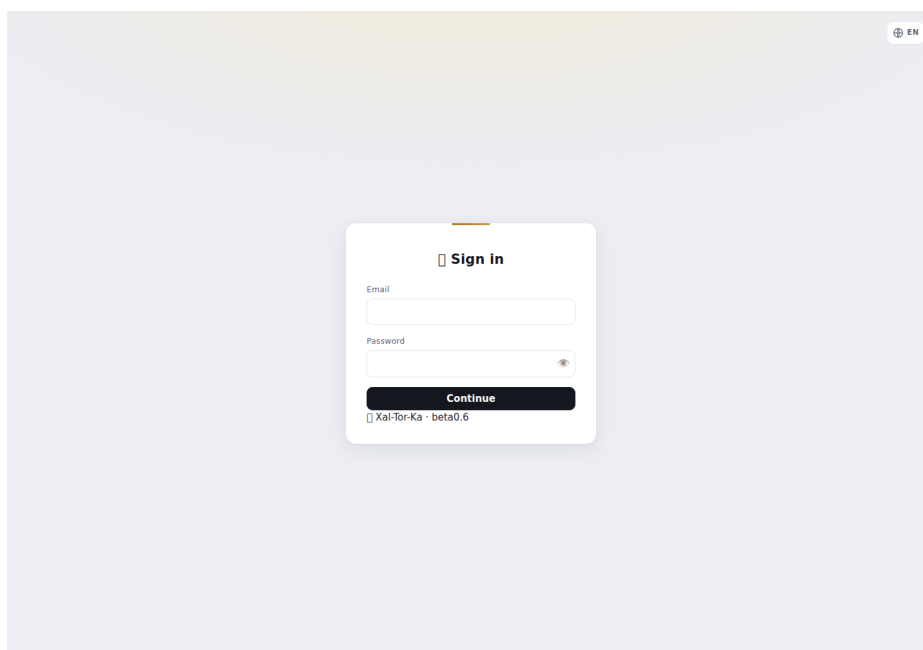


Xai-Tor-Ka — guida pratica (come si fa)

Ricettario operativo: le cose che si fanno più spesso, passo-passo. Presuppone un'istanza già installata (vedi `INSTALL.md`) e un utente admin. Per il perché e il valore vedi [xaltorka-overview.md](#). Screenshot da installazione dimostrativa.

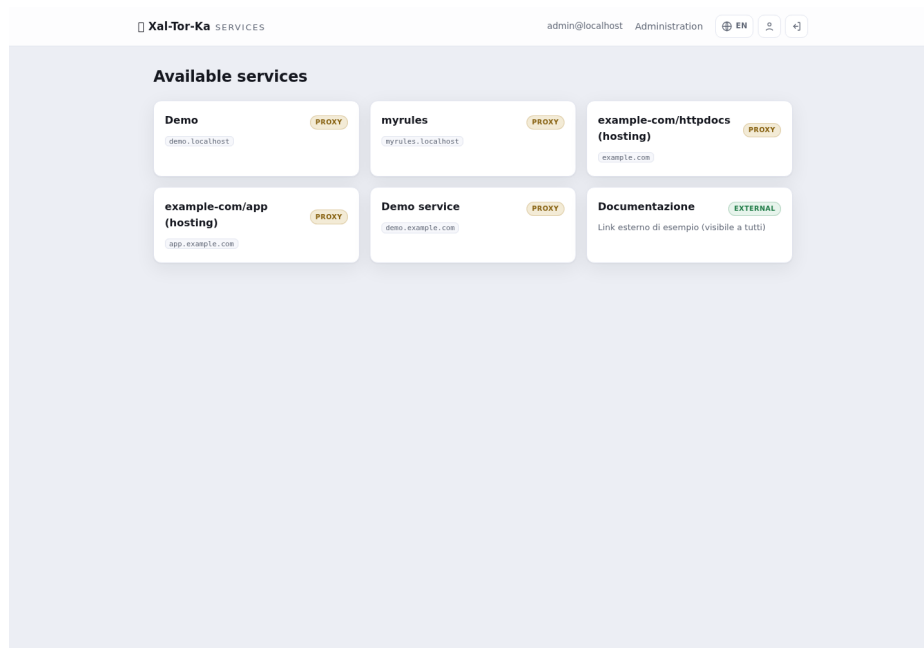
Accesso

Vai all'URL del gateway e fai login. Gli utenti locali usano email + password (+ codice TOTP se il 2FA è attivo); in alternativa un pulsante per ogni provider OIDC configurato. L'area di amministrazione è accessibile solo dagli **IP in whitelist**.



Login

Dopo il login vedi il **catalogo dei servizi** che ti sono visibili. La voce **Administration** in alto porta al pannello di gestione.



Catalogo servizi

Ricetta 1 — Pubblicare un servizio esistente (reverse-proxy)

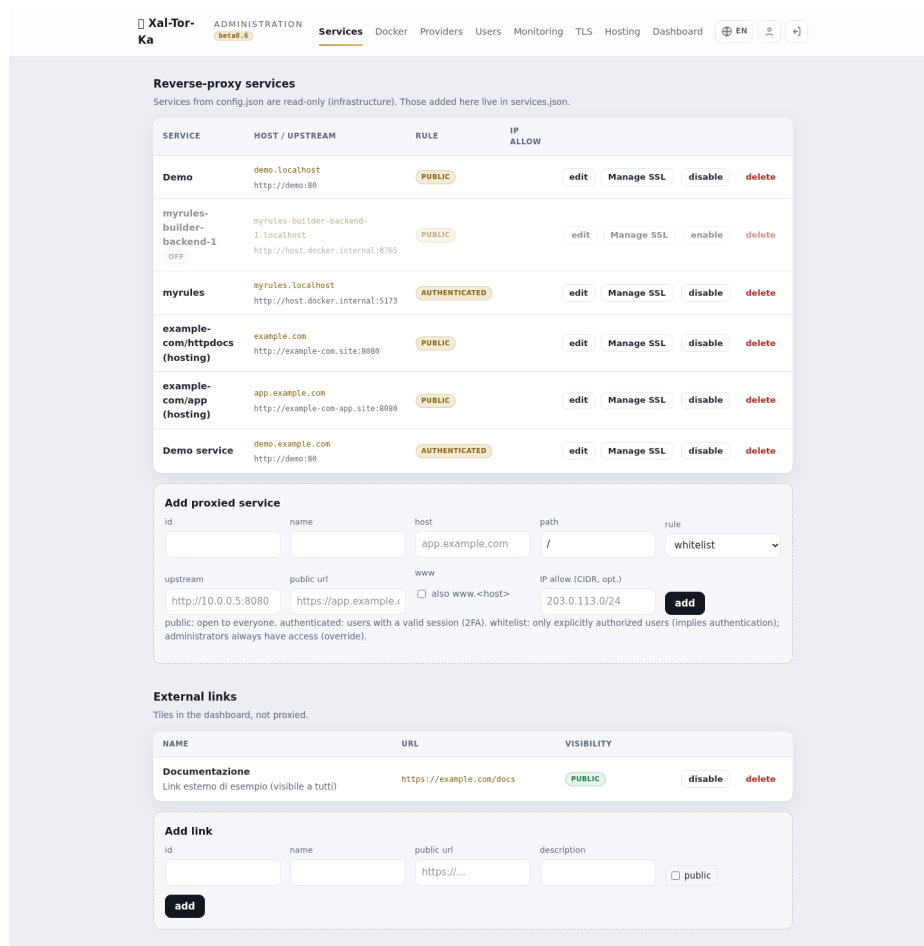
Hai già un servizio che gira (un container, un backend interno) e vuoi metterlo dietro il gate.

1. Administration → Services → Add backend.

2. Compila:

- **Host pubblico:** il dominio da cui si accede (es. app.example.com).
- **Upstream:** l'indirizzo interno del servizio (es. http://mio-servizio:8080; anche un servizio sulla rete Docker interna, non solo sulle reti del computer).
- **Regola:** public (aperto), authenticated (richiede login: entra qualunque utente del gate), authorized (entra **solo** chi abiliti nel tab Accesso del servizio).

3. Salva. Il backend compare nell'elenco e il routing NGINX si aggiorna.



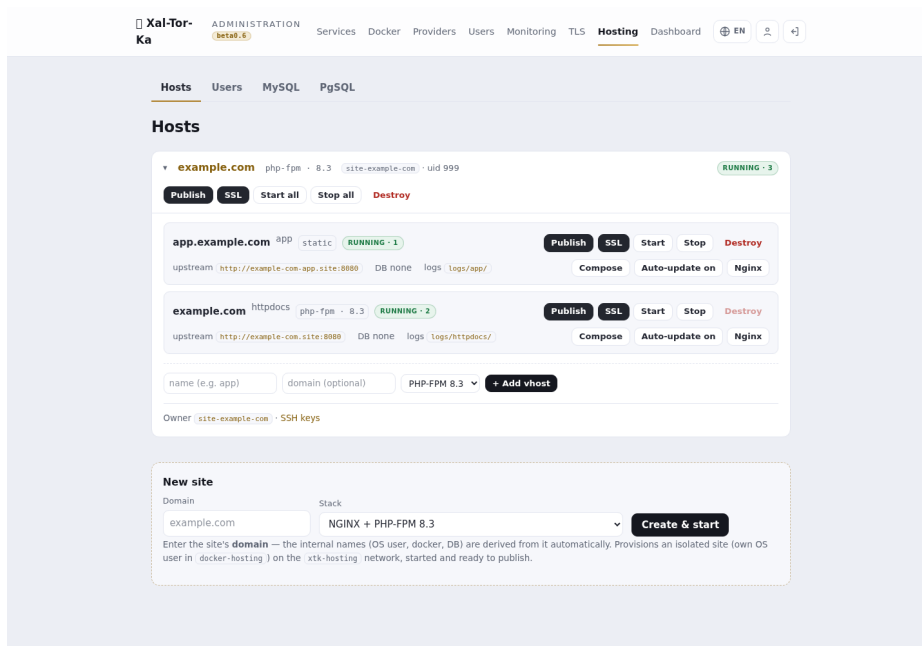
Gestione backend

Poi dai un certificato all'host (Ricetta 4) e punta il DNS del dominio al gateway.

Ricetta 2 — Creare un sito in hosting (Docker)

Xal-Tor-Ka provisiona un sito isolato da zero: utente di sistema dedicato + Docker, sulla rete xtk-hosting.

1. **Administration** → **Hosting** → riquadro **New site** in fondo.
2. Inserisci il **dominio** (es. example.com) e scegli lo **stack**: NGINX + PHP-FPM (8.1/8.2/8.3), con MySQL o PostgreSQL condiviso, statico, o custom (scrivi tu il compose).
3. **Create & start**. Nomi interni (utente OS, container, DB) derivati dal dominio.



Piattaforma hosting

Il sito appare come scheda con il suo vhost httpdocs già avviato. Da qui gestisci Start/Stop, Compose, Nginx, e l'accesso.

Ricetta 3 — Aggiungere un sottodominio (vhost) e pubblicarlo

Un sito può avere più vhost (es. app., api.), ognuno nella sua Docker.

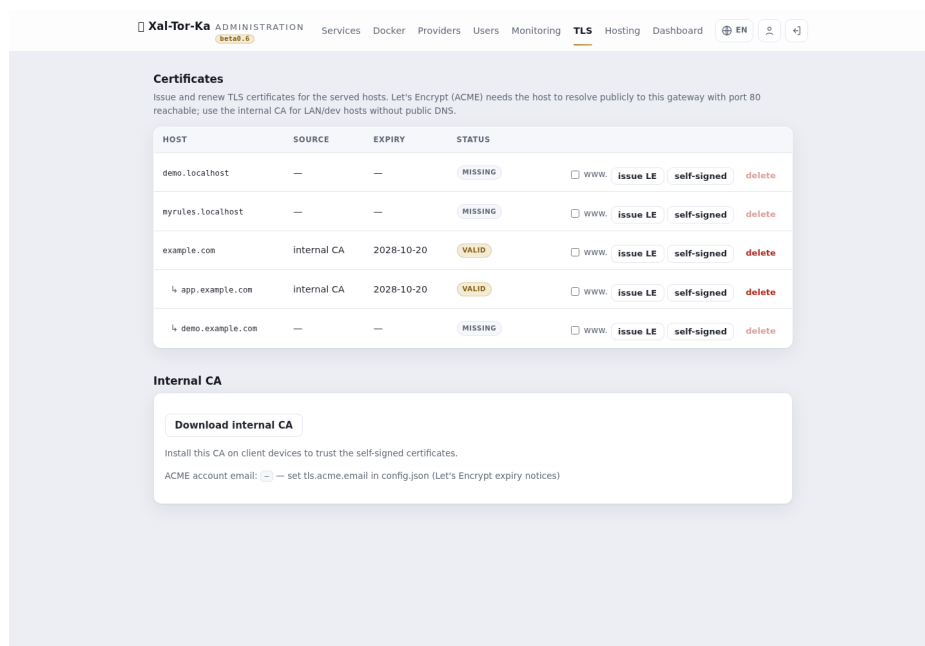
1. Nella scheda del sito, riga **+ Add vhost**: nome (es. app), dominio pubblico opzionale (es. app.example.com), stack.
2. Il vhost parte con il suo container.
3. **Publish**: apri il dialog del vhost, conferma l'**host pubblico** e la **regola** → crea il backend reverse-proxy verso quel vhost.
4. (Poi) **SSL** → emetti il certificato (Ricetta 4).

Il pulsante **Publish** diventa verde quando il servizio è attivo, rosso se disabilitato, nero se non ancora pubblicato. Idem **SSL** per lo stato del certificato.

Ricetta 4 — Emettere un certificato TLS

Administration → **TLS**. La pagina elenca gli **host serviti** (quelli con un backend). I sottodomini compaiono annidati sotto il dominio padre.

- **issue LE** (Let's Encrypt): per host **pubblici** che risolvono al gateway con la porta 80 raggiungibile. La spunta **www** aggiunge anche `www.<host>`.
- **self-signed**: per host **LAN/dev** senza DNS pubblico; usa la **CA interna** (scaricabile in fondo alla pagina, da installare sui client per farla fidare).



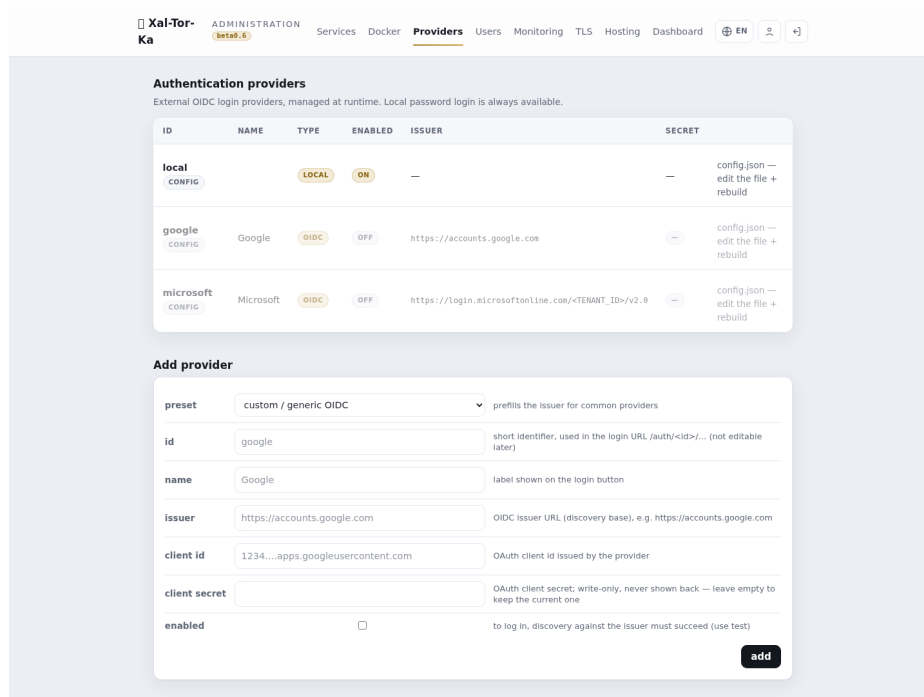
Certificati TLS

Nota: il certificato **segue il backend**. Se un host non compare in TLS, prima pubblicalo (Ricetta 1/3). Per l'ACME il container può anche essere spento: la challenge la risponde il gateway.

Ricetta 5 — Aggiungere un provider OIDC (SSO)

Administration → **Providers** → **Add**.

1. Scegli un **preset** (Google, Microsoft) o custom.
2. Compila **id** (mnemonico), **name** (etichetta sul pulsante), **issuer**, **client_id**, **client_secret**, e spunta **enabled**.
3. Salva. Comparirà un pulsante di login dedicato nella pagina di accesso.



Provider OIDC

Ricetta 6 — LDAP / Active Directory

Per autenticare con gli account di dominio: configura la fonte LDAP (bind LDAPS/StartTLS, template del DN, base DN). Il login locale prova prima le credenziali locali, poi fa fallback su LDAP — un bind riuscito completa l'autenticazione. Gli account di **Active Directory** funzionano via bind LDAPS al domain controller (gli account **locali Windows** no: il SAM non è esposto a un gate Linux). Dettagli in *docs/next-gen-auth-sources.md*.

Ricetta 7 — Database condivisi e Adminer

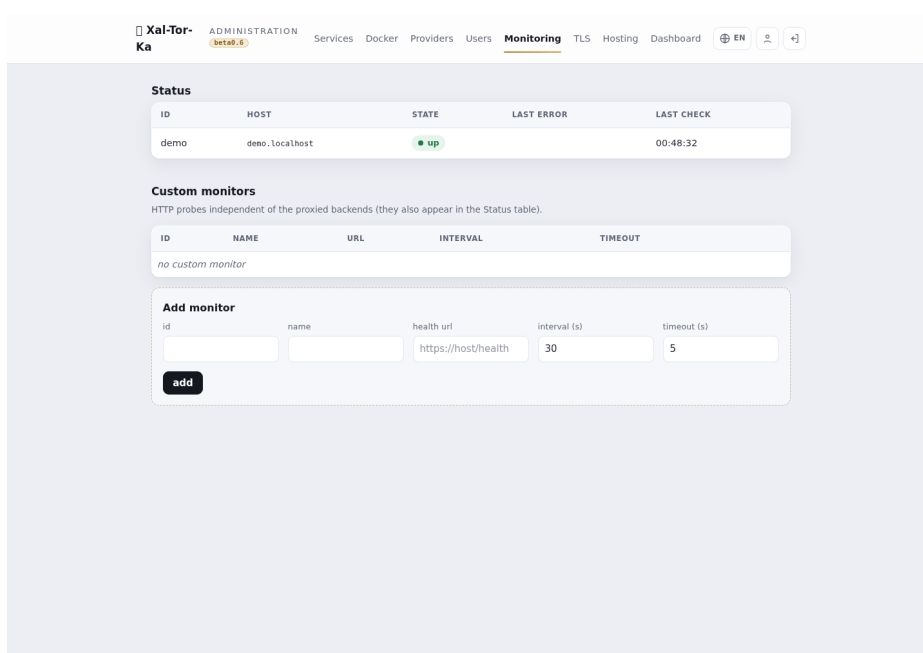
Negli stack hosting puoi attaccare un **MySQL** o **PgSQL** **condiviso**: le tab **MySQL/PgSQL** in Hosting gestiscono le istanze condivise; a ogni sito che lo richiede viene assegnato un DB con utente isolato (nomi/credenziali generati). Un **Adminer** effimero permette di ispezionare i DB.

Ricetta 8 — Accesso SFTP/SSH al sito

Ogni sito ha un utente di sistema con **chroot** per SFTP/SCP (porta dedicata). Dalla scheda del sito → **SSH keys** (o **Owner**): genera/gestisci le **chiavi pubbliche** dell'utente (usabili al posto della password), con download del file combo. Le chiavi si incollano/modificano dal pannello.

Ricetta 9 — Blindare l'admin e le notifiche remote

- **Whitelist IP admin**: imposta gli IP/CIDR autorizzati all'area di amministrazione (il tuo IP pubblico/VPN in produzione). Tutto il resto resta fuori dall'admin.
- **Notifiche/controllo remoto** (opzionale): configura un bot **Telegram** e/o **SMTP/IMAP** per ricevere log di sistema a distanza e mandare **comandi vettati** (allow-list mittenti + comandi; email firmate DKIM). Utile come log-system remoto e per operazioni base senza aprire il pannello.



Monitoraggio

Ricetta 10 — Proteggere path specifici (auth per-path)

Un sito può restare **pubblico** ma con singoli path/file dietro autenticazione — es. mettere wp-login.php e /wp-admin/ dietro login (o Google), lasciando il resto aperto. Difesa in profondità: i bot non raggiungono nemmeno la pagina di login.

1. **Administration** → **Services** → **Modifica** il servizio.
 2. Nella sezione **Regole per path** aggiungi righe: **path** + **match** (esatto = per un file, prefisso per una cartella) + **regola** (authenticated/authorized).
 3. Salva. Il resto di / mantiene la regola principale; la riga più specifica vince. Funziona anche per i siti in hosting (l'upstream resta quello gestito).
-

Ricetta 11 — Difesa dai brute-force (fail2ban)

Il gate scrive un log dei fallimenti di autenticazione; un jail fail2ban banna al **firewall** gli IP che insistono. Difesa a strati: rate-limit in RAM nel gate + ban dell'IP.

- Attivato il jail, in **Administration** → **Hosting** → **System**, il pannello **Firewall — fail2ban** mostra gli **IP bannati** e permette lo **Unban** dall'admin, senza SSH.
 - I ban colpiscono solo le porte web (80/443), **mai SSH**; gli **IP admin e la LAN sono in whitelist** (anti-lockout). Il ban avviene in prerouting nftables, così è efficace anche col gate in container.
-

Ricetta 12 — Aggiornamenti del sistema operativo

Administration → **Hosting** → **System** elenca gli aggiornamenti OS disponibili sull'host (controllo **read-only** via l'agente vettato).

1. Seleziona i pacchetti (o Select security) e **Apply selected** — l'applicazione è admin-gated e **non riavvia mai** da sola.
 2. **Hold** blocca un pacchetto a una versione (no-update); **Release** lo sblocca.
-

Ricetta 13 — WAF (Web Application Firewall)

Metti **ModSecurity + OWASP CRS** davanti a un servizio: blocca SQLi, XSS, path-traversal, scanner. Toggle **per-servizio**.

1. **Administration** → **Services** → **Modifica** il servizio → sezione **WAF**.
2. **Abilita WAF** e scegli la **modalità: Detection-only** (logga, per il rodaggio) o **Blocking** (risponde 403). Parti sempre in Detection-only per tarare i falsi positivi.
3. Sfoghi per-vhost quando una regola dà noie (il WAF è per natura fonte di falsi positivi):
 - **Disabled rules**: spegni singole regole CRS per ID (es. 942100 941110).
 - **Bypass IPs**: IP/CIDR che saltano del tutto il WAF (partner, monitor).
 - **Custom rules (avanzato)**: direttive ModSecurity grezze — es. disabilitare il motore su un path: `SecRule REQUEST_URI "@beginsWith /api/upload" "id:9009500,phase:1,pass,nolog,ctl:ruleEngine=Off"`.
4. Salva. La config è validata con `nginx -t` al reload (config precedente mantenuta se invalida).

Richiede l'immagine nginx con ModSecurity (deploy con `--build`). I falsi positivi si tarano in Detection-only guardando gli eventi, poi si passa a Blocking.

Ricetta 14 — App Laravel (stack pronto)

Metti online un'app **Laravel** senza preparare a mano estensioni o webroot: lo stack **laravel** porta un'immagine php con `pdo_pgsql/pdo_mysql/bcmath/gd/zip/intl/opcode` + **composer**, e serve `public/` come webroot (front-controller).

1. **Hosting** → **New site** (o + **Add vhost**) → stack **NGINX + Laravel (PHP 8.3)**. Il docroot punta automaticamente a `<vhost>/public/`.
2. **DB**: allega un DB condiviso (pgsql/mysql) dalla scheda del sito → utente e DB **isolati**, la connessione arriva nel container via `db.env`. (PostGIS: `CREATE EXTENSION postgis` quando serve.)
3. **Carica il codice** via SCP/SFTP nella cartella del vhost — deve esistere `<vhost>/public/index.php` (l'app intera nel docroot, webroot = `public/`).
4. **Installa e migra**: `composer install --no-dev --optimize-autoloader`, poi `php artisan key:generate --force` e `php artisan migrate --force` (composer è già a bordo; in alternativa spedisci `vendor/` via SCP). Crea scrivibili `storage/` e `bootstrap/cache`.

5. **Pubblica** (Publish) + **TLS** (LE).

Lo stack php-fpm non ha pdo_pgsql → Laravel su postgres darebbe «could not find driver»: lo stack laravel lo risolve bakando le estensioni. Dettagli tecnici in [laravel-stack.md](#).

Ricetta 15 — Moduli PHP à-la-carte

Aggiungi estensioni PHP a un vhost (redis, imagick, ldap, mongodb...) **senza rebuild** e senza conoscere la sintassi Docker. Lo stack php usa l'immagine **xtk-php**: il set base è già cotto, gli extra scelti si materializzano al boot.

1. **Hosting** → **scheda del sito**, sul vhost (php-fpm/laravel) premi «**Moduli PHP**».
2. Spunta i moduli dalla **checklist** (sono già spuntati quelli attivi) → **Salva & applica**.
3. Il container php viene ricreato: i moduli scelti sono compilati e caricati al boot — nessun rebuild dell'immagine, nessun downtime per gli altri vhost del sito.

I moduli sono **allow-listati** lato agente (mai testo libero): un nome fuori lista rifiuta l'intera richiesta. Il set base (pdo_pgsql pdo_mysql bcmath gd zip intl opcache pcntl) è sempre presente.

Ricetta 16 — Log / Criticità (osservabilità)

Vedi in un colpo d'occhio cosa non va: eventi aggregati da più sorgenti e classificati per gravità.

1. **Hosting** → **Log**. La pagina aggrega gli eventi recenti da **journal dell'agente** (esiti dei comandi vettati), **nginx per-vhost** (risposte 4xx/5xx) e **auth del gate** (accessi negati, login falliti).
2. Ogni evento ha un **livello**: **INFO** (normale), **ALERT** (da tenere d'occhio), **CRITICAL** (rotto). Filtra con **TUTTO / INFO+ / ALERT+ / CRITICAL+** (il + = quel livello e superiori).
3. Colonne: livello · sorgente · sito · messaggio · quando. Per un triage veloce parti da **CRITICAL+**.

Sola lettura, nessuna azione distruttiva. I dati vengono dal comando agente diagnostica (read-only). WAF ModSecurity / ACME / fail2ban si aggiungono come sorgenti in evoluzione.

Ricetta 17 — Impostazioni PHP (php.ini / php-fpm)

Regola i limiti PHP di un vhost (dimensione upload, memoria, tempo d'esecuzione...) e, se serve, scrivi direttive avanzate — **senza rebuild** e senza editare file dentro il container. Stesso motore à-la-carte dei moduli: le impostazioni si materializzano come drop-in conf.d al boot.

1. **Hosting → scheda del sito**, sul vhost (php-fpm/laravel) premi «**Moduli PHP**», poi scorri alla sezione «**Impostazioni PHP**».
2. Compila i **limiti comuni** che ti servono — upload_max_filesize, post_max_size, memory_limit, max_file_uploads, max_execution_time. Lascia un campo **vuoto** per il default. Le dimensioni si scrivono 64M, 512M, 1G.
3. (Avanzato) Nel frame «**Direttive php.ini**» aggiungi direttive libere, una per riga (es. opcache.jit=tracing). Nel frame «**Direttive pool php-fpm**» aggiungi direttive di pool [www] (es. pm.max_children = 10).
4. **Salva & applica**: il container php viene ricreato e le impostazioni caricate al boot.

I limiti comuni sono **validati** per pattern lato agente (mai testo libero eseguito). I frame avanzati vengono **scritti come file** (mai eval); il pool php-fpm è validato con php-fpm -t: se una direttiva lo rompe, il drop-in viene scartato e il container parte comunque. Le impostazioni vivono nel vhost (.xhk-stack) e sopravvivono a ricreazioni e redeploy.

Ricetta 18 — Login con codice monouso (passwordless)

Fai accedere un utente con un **codice usa-e-getta** al posto della password. Il codice è un **primo fattore** (chi ha il 2FA lo completa comunque dopo). **Opt-in**, spento di default.

1. In config.json abilita:

```
"one_time_code": { "enabled": true, "channel": "spool",  
                  "ttl_minutes": 10, "code_length": 6 }
```

make rebuild (è config di avvio). Sulla pagina di login compare «**Accedi con un codice monouso**».

2. L'utente inserisce l'email → il gate genera un codice e lo **consegna** secondo il channel:
 - **spool** (default, quando non c'è ancora un account SMTP): il codice viene **scritto in coda** data/otp-queue.jsonl **con l'IP**

del richiedente, timestamp e scadenza. Lo leggi da lì e lo recapiti a mano (o lo usi tu per i test). (La coda è in *data/*, *gitignored*.)

- **email / sms: fase 2** — invio via SMTP (transport notify) / API SMS, quando il cliente del servizio fornirà le credenziali. Finché non configuri, il codice va comunque in coda.

3. L'utente inserisce il codice → **entra**. Il codice è **monouso** (consumato) e **scade** dopo `ttl_minutes`.

Sicurezza: risposta **generica** (non rivela se l'email esiste), codice **solo per utenti reali**, **hashato** a riposo, **usa-e-getta**, a tempo, con **cooldown** anti-spam per email. La coda contiene i codici in chiaro (serve per recapitarli): tienila protetta finché usi `pool`.

Ricetta 19 — Pubblicare un servizio su un path di un dominio esistente

Hai un servizio interno (una docker con la sua porta, oppure un vhost dell'hosting) e vuoi esporlo su `miominio.it/strumento` **senza toccare l'hosting del dominio**. Il servizio resta una **voce propria** nei Servizi: si pubblica, si protegge e si rimuove da solo.

1. **Services → Add backend**:

- **Host**: il dominio che già esiste (es. `sfs.it`) — lo stesso del sito.
- **Path**: il path pubblico (es. `/strumento`).
- **Upstream**: dove gira davvero (es. `http://mia-docker:8080`).
- **Regola**: `public`, `authenticated` o `authorized` (vedi il riquadro sotto).

2. Salva. Il dominio ora ha **due voci**: il sito (`/`) e il tuo servizio (`/strumento`), indipendenti. Il path viene **inoltrato** all'upstream (non viene tolto): se l'app ha bisogno di sapere il prefisso, usa `X-Forwarded-Prefix` in **Modifica → NGINX custom**.

3. Il certificato TLS resta **uno per dominio**: se il sito ce l'ha già, il servizio lo eredita.

Chi comanda sull'host. Più servizi possono condividere lo stesso `hostname`: `nginx` riceve **un solo blocco** per dominio. Le impostazioni di dominio — certificato, `www.`, WAF, `client_max_body_size`, «NGINX custom (server)» — le porta il **servizio primario**, cioè quello che possiede `/`. Gli altri contribuiscono **solo la propria location**: il loro «NGINX custom (location)» vale **soltanto per il loro path** e non tocca gli altri servizi del dominio (utile se ci inietti un header segreto: resta confinato). Due servizi con **stesso host e stesso path** non sono ammessi: vince il primo e il duplicato viene ignorato.

⚠ **Effetto da conoscere:** appena **un solo** servizio del dominio non è `public`, il gate inizia a servire su quel dominio i suoi path riservati — `/login`, `/logout`, `/auth/`, `/listing`, `/_xtk/` — perché il redirect al login deve funzionare lì. Sono pochi e con prefisso `_xtk` proprio per pestare i piedi il meno possibile: il sito **mantiene il suo** `/assets/` e tutto il resto. Prima di proteggere un path su un sito già online, controlla di non usare tu uno di quei cinque.

☐ **authenticated non è «solo il mio utente».** `authenticated` significa qualunque utente del gate con una sessione valida — non guarda le autorizzazioni per-servizio. Se vuoi che entri **solo** chi hai deciso, usa `authorized` e spunta gli utenti nel tab **Accesso** della scheda del servizio. È la differenza fra «serve un login» e «serve quel login».

Ricetta 20 — Il listing pubblico dei servizi (`/listing`)

La pagina `/listing` è la vetrina dei servizi del server. Da `beta0.12` decidi tu **cosa** mostrare e **come**.

1. **Services → Modifica** il servizio:
 - «**Esponi nel listing**»: togli la spunta per tenerlo fuori dalla vetrina (strumenti interni).
 - **Descrizione**: accetta **Markdown** (grassetti, elenchi, link). Viene **sanitizzata** prima di essere mostrata: niente script o HTML pericoloso, anche se lo incolli.
 - **Immagine di anteprima**: caricane una — compare sulla card del servizio.
2. I servizi di uno stesso **multidominio** vengono **raggruppati** in un unico blocco, invece di apparire come schede sparse.

Nota: la descrizione è pensata per gli **umani** che arrivano sul server. La sanitizzazione è attiva sempre e non è disattivabile: il listing è pubblico.

Ricetta 21 — Emissione Let's Encrypt: cosa succede mentre aspetti

Premendo «**Emetti LE**» l'emissione può durare parecchi secondi (ACME deve verificare il dominio). Da `beta0.12` compare un **overlay** che dice «Emissione certificato Let's Encrypt per `<host>...`» finché l'operazione non si chiude: non è bloccato, sta lavorando. Non ricaricare la pagina a metà.

Ricetta 22 — Chi entra: regole d'accesso e permessi per directory

La scheda di un servizio è divisa in quattro tab — **Generale** · **Accesso** · **NGINX** · **Sicurezza**. Tutto ciò che riguarda «chi entra» sta in **Accesso**.

Le tre regole

Regola	Chi entra
Pubblico	chiunque, senza login.
Autenticato	serve un account sul gateway: entra chiunque abbia fatto login.
Autorizzati	serve un account e l'abilitazione a questo servizio : entra solo chi spunti.

□ La distinzione che conta è **autenticazione** ≠ **autorizzazione**. «Autenticato» chiede solo chi sei; se ti serve «lo vede **solo** quella persona», la regola è **Autorizzati**. Gli **amministratori entrano sempre**, e l'**allow-list IP** è indipendente dalla regola: vale per tutte e tre, Pubblico incluso.

(«Autorizzati» si chiamava *whitelist*. Rinominata perché il prodotto ha già un'allow-list di IP e la stessa parola indicava due cose diverse. I file scritti prima continuano a funzionare.)

Abilitare le persone

Nel tab **Accesso** c'è la lista **Utenti abilitati**: spunta chi può usare il servizio. La lista resta **visibile anche quando la regola non la usa** (spenta, con una nota): così puoi preparare gli accessi prima di passare ad «Autorizzati», invece di cambiare regola e poi riabilitare tutti. Gli amministratori compaiono marcati e non selezionabili — entrano comunque, e nasconderli racconterebbe una cosa falsa su chi può accedere. Con molti utenti, usa «**+ Aggiungi utente**» e il filtro per email.

Permessi per directory (l'ereditarietà)

Nella tabella **Regole per path** ogni riga ha la colonna **Utenti** con due scelte: - **Eredita** (predefinito) — quella directory usa la lista del servizio. - **Propri** — quella directory ha la **sua** lista: apri «scegli utenti» e spunta chi vale **lì**.

L'ereditarietà **scende**: una sottocartella che eredita prende la lista dell'**antenato più vicino** che ne ha una propria, e se nessun antenato la definisce ricade sul servizio. Esempio:

```
/ → lista del servizio
/riservato (Propri) → lista A
/riservato/doc → eredita ⇒ lista A (l'antenato più vicino)
/riservato/x (Propri) → lista B (la propria vince)
/altro → eredita ⇒ lista del servizio
```

Un prefisso parziale **non** è un antenato: /riservatox non eredita da /riservato.

Attenzione: una directory con lista **propria** e **nessun utente spuntato** è chiusa a tutti (tranne gli amministratori) — non aperta. È voluto: sull'accesso, il default deve sbagliare verso il chiuso.

Riferimento rapido

Voglio...	Vai a
Mettere auth/HTTPS davanti a un servizio	Services → Add backend, poi TLS
Creare un sito nuovo	Hosting → New site
Creare un'app Laravel	Hosting → New site → stack Laravel
Aggiungere un sottodominio	Hosting → scheda sito → + Add vhost → Publish
Moduli o impostazioni PHP di un sito	Hosting → scheda sito → Moduli PHP
Login con codice usa-e-getta	config one_time_code → «Accedi con un codice»
Un certificato	TLS → issue LE / self-signed
SSO aziendale	Providers (OIDC) / config LDAP
Accesso ai file del sito	Hosting → scheda sito → SSH keys (SFTP)
Chi può entrare nell'admin	Whitelist IP admin
Proteggere wp-login/una cartella	Services → Modifica → Accesso → Regole per path
Far entrare solo certe persone	Accesso → regola «Autorizzati» + spunta gli utenti
Utenti diversi su una sottocartella	Regole per path → colonna Utenti → «Propri»

Voglio...	Vai a
Pubblicare un servizio su dominio/path	Services → Add backend (host esistente + path)
Nascondere un servizio dalla vetrina	Services → Modifica → «Esponi nel listing»
Bloccare i brute-force	fail2ban (jail) → Hosting → System (IP bannati/Unban)
Aggiornare l'OS dell'host	Hosting → System → Apply selected
Firewall applicativo (SQLi/XSS)	Services → Modifica → WAF (Detection-only → Blocking)
Aggiungere estensioni PHP a un sito	Hosting → scheda sito → vhost → Moduli PHP
Vedere errori/criticità aggregati	Hosting → Log (filtra CRITICAL+)

Xal-Tor-Ka è software di SFS.it. Screenshot da installazione dimostrativa con dati di esempio.